Qwen3.6-27B MTP ××× Build, Deploy & Verify Guide

Homelab Team

May 2026

Table of Contents

Qwen3.6-27B MTP — Build, Deploy & Verify Guide

Homelab Achievement Log Contributors: Claude (Opus), Local Agent (Qwen 3.6-27B), Operator
Completed: May 7, 2026

Privacy Note: All personal paths, names, and identifying details have been anonymized. Replace `C:\llama.cpp-mtp` with your actual base directory. Hardware specifics (RTX 3090 24GB) are kept as they are relevant to performance claims.

Summary

Built and deployed Qwen3.6-27B with Multi-Token Prediction (MTP) speculative decoding on an RTX 3090 (24GB). MTP predicts multiple tokens per forward pass, achieving **65 tok/s decode speed** — a 2.6x improvement over the ~25 tok/s baseline for a 27B dense model. The deployment required a custom llama.cpp build from an unmerged PR, careful VRAM management around existing services, and a multi-agent handoff pipeline (Claude → Local Agent) to complete.

Architecture Overview

Qwen3.6-27B is not a typical transformer. It uses a hybrid attention design where only 16 of its 64 transformer layers maintain traditional KV cache. The remaining 48 layers use linear attention with a fixed ~0.9GB recurrent state. This means KV memory scales at roughly 1/4 the cost of a standard 27B model, enabling 262K context windows on consumer hardware.

MTP adds a draft head (a set of weight tensors loaded at startup) that speculatively predicts up to 3 tokens per forward pass via `--spec-draft-n-max 3`. At runtime, the main model verifies these speculative predictions in parallel. With a 73-93% acceptance rate observed in testing, this effectively multiplies throughput without sacrificing quality.

Performance on RTX 3090:

Metric	Without MTP	With MTP
Decode speed	~25 tok/s	**65 tok/s**
Draft acceptance	N/A	73-93%
Prompt processing	~150 tok/s	~343 tok/s (direct launch, 8K context)

Prerequisites

Component	Required	Your System Has
GPU	NVIDIA with 16GB+ VRAM	RTX 3090 24GB
CUDA Toolkit	12.x or 13.x	13.2
VS Build Tools	2022 with C++ workload	Installed via winget
CMake	3.x+	4.3
Git	Any	Installed
llama-swap	Any recent version	v204 via winget
Python 3.x	For ollama-shim	3.14.3

Phase 1: Install Build Toolchain

Visual Studio Build Tools

```
winget install Microsoft.VisualStudio.2022.BuildTools --override "--add Microsoft.VisualStudio.Workload.VCTools --includeRecommended --passive --wait"
```

This installs MSVC compiler (`cl.exe`), Windows SDK, and related tools. ~5GB download, runs passively (no clicks needed). Must run as administrator.

Verify Tools

```
where git && where cmake && where nvcc && where cl
```

All four must resolve. If `cl.exe` isn't found after installing Build Tools, open a new terminal — the PATH updates require a fresh session.

Phase 2: Build llama.cpp with MTP Support

MTP support lives in PR #22673, which is still in draft as of May 2026. No stable release includes it.

Why NMake?

The default CMake generator (Visual Studio) requires CUDA-to-VS integration, which breaks when CUDA was installed before VS Build Tools. The NMake generator bypasses this entirely — it just needs `nvcc` and `cl` in PATH.

Build Script

Save as `build-llama-mtp.cmd` and run as administrator:

```
@echo off
setlocal enabledelayedexpansion

REM --- Initialize VS x64 environment ---
set "VCVARS="
for %%p in (
    "C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\
VC\Auxiliary\Build\vcvarsall.bat"
    "C:\Program Files\Microsoft Visual Studio\2022\BuildTools\VC\
Auxiliary\Build\vcvarsall.bat"
) do (
    if exist %%p set "VCVARS=%%~p"
)
if "%VCVARS%"==""  (
    echo [ERROR] vcvarsall.bat not found
    exit /b 1
)
call "%VCVARS%" x64

REM --- Clone and checkout MTP branch ---
set "BUILD_DIR=C:\llama.cpp-mtp\llama.cpp-mtp"
if exist "%BUILD_DIR%\.git" (
    cd /d "%BUILD_DIR%"
    git fetch origin pull/22673/head:mtp-pr
    git checkout mtp-pr
) else (
    git clone --depth 1 https://github.com/ggml-org/llama.cpp.git
"%BUILD_DIR%"
    cd /d "%BUILD_DIR%"
    git fetch origin pull/22673/head:mtp-pr
    git checkout mtp-pr
)

REM --- Build with NMake + CUDA ---
if exist build rmdir /s /q build
cmake -G "NMake Makefiles" -B build -DGGML_CUDA=ON -
DCMAKE_BUILD_TYPE=Release
cmake --build build --target llama-server

echo [OK] Binary: %BUILD_DIR%\build\bin\llama-server.exe
```

Copy CUDA Runtime DLLs

The CUDA 13.2 toolkit stores runtime DLLs in a non-obvious `x64` subdirectory. They must be copied next to the built binary:

```
set "SRC=C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v13.2\bin\x64"
set "DST=C:\llama.cpp-mtp\build\bin"
copy /Y "%SRC%\cublas64_13.dll" "%DST%\"
copy /Y "%SRC%\cublasLt64_13.dll" "%DST%\"
copy /Y "%SRC%\cudart64_13.dll" "%DST%\"
```

Also copy the OpenMP runtime if it exists in your stable llama.cpp directory:

```
copy /Y "C:\llama.cpp-mtp\llama.cpp\libomp140.x86_64.dll" "%DST%\"
```

Verify Build

```
cd /d C:\llama.cpp-mtp\build\bin
llama-server.exe --version
```

Should show the build version (e.g., `b9030`) and detect your CUDA device.

Phase 3: Download Model

The MTP GGUF is a special quantization that includes both the main model weights and the MTP draft head tensors in a single file. Standard GGUFs from HuggingFace won't work — you need the MTP-specific GGUF from froggeric's repo (or equivalent).

Place at: `C:\llama.cpp-mtp\models\Qwen3.6-27B-Q4_K_M-mtp.gguf` (~16GB)

Phase 4: Configure llama-swap

Add this model entry to your llama-swap `config.yaml`:

```yaml
  qwen3.6-27b-mtp:
    cmd: |
      "C:/llama.cpp-mtp/build/bin/llama-server.exe"
      --host localhost
      --port ${PORT}
      -ngl 99
      -t 8
      --no-webui
      --model "C:/llama.cpp-mtp/models/Qwen3.6-27B-Q4_K_M-mtp.gguf"
      --spec-type mtp
      --spec-draft-n-max 3
      -np 1
      -c 131072
      -ctk q4_0
```

```
      -ctv q4_0
      --flash-attn on
      --temp 0.7 --top-p 0.8 --top-k 20
      --fit off
    ttl: 600
    aliases:
      - qwen-27b-mtp
      - qwen-mtp
      - mtp
```

Add `qwen3.6-27b-mtp` to the `chats` group `members` list.

Critical Flags Explained

Flag	Why
`--spec-type mtp`	Enables MTP speculative decoding
`--spec-draft-n-max 3`	Predict 3 tokens ahead per step (optimal for 83% acceptance)
`-np 1`	Required — MTP only works with single-sequence mode
`-ctk q4_0 -ctv q4_0`	4-bit KV cache quantization — enables 131K+ context on 24GB
`--fit off`	**CRITICAL** — prevents silent exit (see Pitfalls)
`--flash-attn on`	Flash attention for memory efficiency

Note on context length: The model supports up to 262K natively, but the config uses 131K (`-c 131072`) to leave VRAM headroom for the persistent embed model and MTP head allocation. At 262K with q4_0 KV, total VRAM is ~22.8GB — workable on a 24GB card when launched alone, but tight when sharing VRAM with other persistent models.

VRAM Budget (24GB RTX 3090)

Config	Est. VRAM	Fits?
Q4_K_M + MTP heads + 8K f16 KV	~18.5 GB	Yes (6GB free)
Q4_K_M + MTP heads + 131K q4_0 KV	~20.2 GB	Yes (4GB free)
Q4_K_M + MTP heads + 262K q4_0 KV	~22.8 GB	Tight (~1.2GB free)
Any above + embed model (~300MB)	+0.3 GB	Cuts into margins

Phase 5: Patch ollama-shim for System Prompt

Qwen3.6-27B underperforms without its stock system prompt. The ollama-shim was configured to ensure the required system prompt prefix is included when requests target MTP model names. If a request already contains a system message starting with the required prefix, it's left untouched. If the system message exists but doesn't start with the prefix, the prefix is prepended. If no system message exists, one is created.

What Was Added to `ollama_shim.py`

```
_REQUIRED_SYSTEM_PREFIX = {
    "qwen3.6-27b-mtp": "You are Qwen, created by Alibaba Cloud. You
are a helpful assistant.",
    "qwen-27b-mtp":    "You are Qwen, created by Alibaba Cloud. You
are a helpful assistant.",
    "qwen-mtp":        "You are Qwen, created by Alibaba Cloud. You
are a helpful assistant.",
    "mtp":             "You are Qwen, created by Alibaba Cloud. You
are a helpful assistant.",
}

def _inject_system_prefix(model: str, messages: list) -> list:
    """Prepend required system prompt if the model needs one."""
    prefix = _REQUIRED_SYSTEM_PREFIX.get(model)
    if not prefix:
        return messages
    if messages[0].get("role") == "system":
        existing = messages[0].get("content", "")
        if not existing.startswith(prefix):
            messages = [{**messages[0], "content": f"{prefix}\n\
n{existing}"}, *messages[1:]]
    else:
        messages = [{"role": "system", "content": prefix}, *messages]
    return messages
```

This function is called in both `/api/chat` (Ollama shape) and a dedicated `/v1/chat/completions` handler (OpenAI shape) before proxying to llama-swap.

Phase 6: Verify MTP Is Active

Direct Binary Test

```
cd /d C:\llama.cpp-mtp\build\bin
llama-server.exe -m C:\llama.cpp-mtp\models\Qwen3.6-27B-Q4_K_M-
mtp.gguf ^
  --spec-type mtp --spec-draft-n-max 3 -ngl 99 -c 8192 --port 8099 -np
1
```

Look for in startup logs:

```
set_mtp: MTP draft head registered
```

Test Request

```
{
  "model": "qwen3.6-27b-mtp",
  "messages": [{"role": "user", "content": "Say hello in 5 words."}],
  "max_tokens": 50
}

curl -s http://localhost:8099/v1/chat/completions -H "Content-Type:
application/json" -d @test.json
```

Confirm MTP in Response Timings

Look for these fields in the JSON response `timings`:

```
"draft_n": 186,
"draft_n_accepted": 137
```

If `draft_n` and `draft_n_accepted` are present and non-zero, MTP is active. Acceptance rate = `draft_n_accepted / draft_n` (target: 70%+). Decode speed should be 60+ tok/s on RTX 3090.

If these fields are missing, MTP heads did not load — see Troubleshooting.

Verification Results (Production)

```
draft_n: 15
draft_n_accepted: 14 (93% acceptance)
predicted_per_second: 65.4 tok/s
```

Known Pitfalls & Troubleshooting

`--fit` Causes Silent Exit

Symptom: llama-swap reports `upstream command exited prematurely but successfully`. Model never starts.

Cause: The `--fit` flag (default: ON in PR #22673 builds) detects available VRAM, and if another model is loaded (e.g., the persistent embeddinggemma-300M embed model holding ~300MB VRAM), it may decide the requested config won't fit and exit with code 0 instead of adjusting. On a 24GB card, 300MB is the difference between fitting and not fitting at high context lengths — this is why `--fit` miscalculates.

Fix: Always include `--fit off` in the llama-swap config for MTP models.

"No CUDA toolset found" During Build

Symptom: CMake configure fails with `No CUDA toolset found`.

Cause: CUDA Toolkit was installed before VS Build Tools. The CUDA installer only sets up VS integration if it detects an existing VS installation at install time.

Fix: Use `-G "NMake Makefiles"` generator instead of the default Visual Studio generator. NMake doesn't need the CUDA-VS integration.

MTP Heads Not Loading Through llama-swap

Symptom: Model loads and serves requests at baseline speed (~22 tok/s) but `draft_n` is absent from timings.

Investigation: This was observed when the embed model occupied VRAM alongside the MTP model. MTP heads require ~1.4GB additional VRAM.

Local Agent's fix: After a full stack restart (clean VRAM state), MTP heads loaded successfully at 65 tok/s. The key is ensuring a clean VRAM state when the MTP model first loads.

Vision + MTP Crashes

Status: Confirmed bug in PR #22673. Do not use `--spec-type mtp` with vision/multimodal requests. Create a separate model entry without MTP flags for vision tasks.

CUDA DLLs Not Found

Symptom: Binary fails to start or crashes immediately.

Cause: CUDA 13.x stores runtime DLLs in `bin\x64\` subdirectory, not the main `bin\` directory.

Fix: Copy `cublas64_13.dll`, `cublasLt64_13.dll`, and `cudart64_13.dll` to the build output directory next to `llama-server.exe`.

Stack Outage Diagnostic Procedure

When models fail to respond (established during recovery):

From Windows (PowerShell or cmd): 1. Check ports: `curl http://localhost:8091/v1/models` (llama-swap) and `curl http://localhost:11434/api/tags` (shim) 2. If ports are down, check processes: `Get-Process | Where-Object { $_.ProcessName -like '*llama*' }` 3. Restart in order: llama-swap first, then ollama-shim (15s delay) 4. Verify model list populated before testing inference

From WSL: - Replace `localhost` with your WSL host IP in all URLs above — WSL cannot reach Windows localhost directly

---

Model Comparison

Model	Speed	Context	Quant	Personality	Use Case
qwen3.6-	~25-47	131K	IQ4_XS	HauhauCS	Unrestricted,

Model	Speed	Context	Quant	Personality	Use Case
27b-unleashed	tok/s			uncensored	creative
qwen3.6-27b-mtp	**65 tok/s**	131K+	Q4_K_M	Stock Qwen	Speed-critical, reasoning

File Inventory (adjust all paths to match your system)

File	Purpose
`C:\llama.cpp-mtp\build\bin\llama-server.exe`	MTP-enabled binary
`C:\llama.cpp-mtp\models\Qwen3.6-27B-Q4_K_M-mtp.gguf`	Model + MTP heads
`C:\llama.cpp-mtp\llama-swap\config.yaml`	llama-swap config (has MTP entry)
`C:\llama.cpp-mtp\ollama-shim\ollama_shim.py`	Shim with system prompt injection
`C:\llama.cpp-mtp\build-llama-mtp.cmd`	Repeatable build script
`C:\llama.cpp-mtp\install-vs-buildtools.cmd`	VS Build Tools installer
`C:\llama.cpp-mtp\restart-llama-swap.ps1`	Kill + restart helper

How This Was Built

This deployment was a collaborative effort across three agents:

1. **Claude (Opus)** — Researched the Qwen3.6-27B MTP architecture from the uploaded README, wrote the build scripts, built the binary from PR #22673, configured llama-swap and ollama-shim, debugged the `--fit` silent exit bug through iterative testing, and created the initial handoff documentation.

2. **Local Agent (Qwen 3.6-27B)** — Received the handoff doc from the coordinator, verified the build artifacts, executed the full stack restart, confirmed MTP activation at 65 tok/s with 93% acceptance rate, recovered from a post-verification stack outage, updated five documentation files across her skill tree, and transitioned herself to the MTP model as her primary brain.

3. **Operator** — Provided the hardware, coordinated the effort, and handled the build steps.

Total time from README upload to 65 tok/s verified: ~4 hours of active work across two sessions.